# Trabajo práctico integrador

# Lógica y Teoría de Grafos

Martín Moloeznik

Nicolás Paz Reyes

https://paz-reyes.moloeznik.me/

8 de noviembre de 2026

# Índice general

**1. Planteo** **2**

**2. Los datos** **3**
2.1. Búsqueda de una fuente con coordenadas . . . 3
2.2. Contenedores de la Ciudad de Buenos Aires . . . 3
2.3. Calles . . . 3
2.4. Tipo de zona . . . 4
2.5. Columnas generadas . . . 4
2.5.1. Días y quejas . . . 4
2.5.2. La prioridad . . . 4
2.5.3. Cómo se generaron . . . 5

**3. El modelo** **6**
3.1. Columnas . . . 6
3.2. Comparación . . . 6
3.3. Resultado . . . 7

**4. El circuito** **9**
4.1. Calles a cubrir . . . 9
4.2. El camino de unión . . . 9
4.3. Grados impares . . . 9
4.4. Componentes que siguen separadas . . . 10
4.5. El recorrido . . . 10

**5. Capturas del frontend** **11**

**6. Uso de inteligencia artificial** **13**
6.1. Qué herramientas se usaron . . . 13
6.2. Para qué partes se usaron, y para cuáles no . . . 13
6.3. Ejemplos de prompts . . . 13
6.4. Casos en los que la IA se equivocó . . . 13
6.5. Reflexión . . . 13

**7. Limitaciones** **14**

# Capítulo 1

# Planteo

El problema es el recorrido de un camión de residuos. A diferencia de un reparto, donde alcanza con visitar puntos, el camión recorre calles enteras. No todas las calles tienen la misma urgencia: en algunas hay contenedores y quejas, y en otras no hay nada que levantar. Hay que estimar qué tramos conviene recorrer primero y, con esa lista, armar un circuito que los cubra.

El sistema tiene tres partes. Un mapa de calles, con datos medidos donde hay una fuente pública y datos generados donde no la hay. Un clasificador que asigna a cada tramo prioridad alta, media o baja. Un circuito que pasa por todas las calles de prioridad alta y une los tramos sueltos por el camino más corto, en metros. En ese camino entran las calles que haya, sean de prioridad media o baja. Si cambian los datos de una calle, la predicción puede cambiar y el circuito se calcula de nuevo.

La zona es un grafo no dirigido. Cada nodo es una esquina y cada arista es un tramo de calle con su largo en metros. El camión recorre el tramo en cualquier sentido.

La zona de trabajo es un recorte de unos dos kilómetros de lado en el microcentro de Buenos Aires, San Nicolás y el borde de Retiro y Monserrat, con un tope de unas 1500 aristas.

La calle, el largo, el nombre, el tipo de zona y la cantidad de contenedores salen de fuentes públicas. Los días desde la última recolección, el historial de quejas y la prioridad de recolección los genera un script, con reglas fijas. Cerca del 20 % de los tramos queda en alta, el 35 % en media y el 45 % en baja. El corte es un ranking de un puntaje con ruido, así la clase no es una función cerrada de las otras columnas.

El modelo, los datos y el circuito están en Python. El clasificador usa scikit-learn. El grafo de calles se arma con OpenStreetMap y queda guardado. Dijkstra y Hierholzer están implementados en el proyecto. El emparejamiento de costo mínimo del cartero chino lo resuelve rustworkx, con el algoritmo de Edmonds. La API, en FastAPI, carga el clasificador ya ajustado y arma el circuito con las prioridades que predice. El mapa del navegador consume esa API.

Para correr el proyecto en local, los pasos están en `README.md`, secciones Setup y Correr. El comando de arranque es `docker compose -f compose.dev.yaml up --build`.

# Capítulo 2

# Los datos

## 2.1. Búsqueda de una fuente con coordenadas

La búsqueda empezó por Rosario. El portal de datos abiertos de la municipalidad publica un dataset de contenedores de residuos domiciliarios. El 30 de octubre de 2026 el sitio no respondía: daba timeout o error 504, también desde el navegador. No apareció un CSV espejo con coordenadas.

En OpenStreetMap, el centro de Rosario tiene 57 papeleras marcadas. Cubren cerca del 3 % de las calles de un recorte de ese tamaño, así que casi todos los tramos quedarían sin contenedor.

La Plata no publica un inventario de contenedores con latitud y longitud. Hay un dataset genérico de residuos de 2020, sin geometría usable, y un mapa de focos de basura.

## 2.2. Contenedores de la Ciudad de Buenos Aires

La fuente publicada, con coordenadas y licencia Creative Commons Attribution, es la del Gobierno de la Ciudad de Buenos Aires. En Buenos Aires Data, dentro de "Infraestructura de gestión de residuos", el recurso es el de contenedores para residuos húmedos de la zona 1. El archivo es un CSV separado por punto y coma. Cada fila es un contenedor, con latitud y longitud. También trae una geometría en coordenadas planas de la Ciudad. La ubicación que se usa es la latitud y la longitud.

A veces el número viene sin punto decimal, por ejemplo $-3461477$ en lugar de $-34{,}61477$. Al leer el archivo se divide por diez hasta que la latitud cae entre $-90$ y 90 y la longitud entre $-180$ y 180.

La zona 1 llega también a Constitución, San Telmo y Puerto Madero. Ese polígono entero supera las 1500 aristas, así que el mapa queda en el microcentro. Cada contenedor se asigna a la calle más cercana si está a 50 metros o menos. El archivo trae 2292 contenedores y entran 887. Los otros 1405 quedan a más de 50 metros de todas las calles del recorte. El 32 % de los tramos tiene al menos un contenedor y el máximo en un tramo es 9. Si no hay ninguno, la cantidad queda en 0. El tramo guarda el conteo, no la posición de cada contenedor.

## 2.3. Calles

Las calles salen de OpenStreetMap, licencia ODbL. La red es la de autos: sin peatonales, sin ciclovías y sin calles de servicio. El servicio público de Overpass estaba saturado, así que el extracto se baja de la API de mapa de OpenStreetMap y el filtro se aplica después.

El grafo de origen trae sentido de circulación. Para este problema se toma no dirigido: el camión recorre la calle. Se descartan los lazos de una esquina a sí misma. Si entre el mismo par de esquinas quedan dos aristas, queda la más corta. Después queda solo la componente conexa más grande, de modo que entre dos esquinas del mapa siempre hay un camino.

El grafo tiene 986 esquinas y 1257 tramos. Cada tramo tiene un identificador entero, las dos esquinas con el menor número primero, el largo en metros, el tipo de vía y el nombre de la calle cuando OpenStreetMap lo trae. Las coordenadas de las esquinas van en otro archivo. El mismo identificador queda escrito en el grafo.

## 2.4. Tipo de zona

Cada tramo es residencial, comercial o industrial. En el microcentro OpenStreetMap casi no marca manzanas industriales, así que el tipo sale de lo mapeado en un buffer de 40 metros alrededor del tramo.

El puntaje mira locales, usos del suelo comercial, amenities de comercio y edificios comerciales. Hace falta un puntaje de al menos 6 para comercial. Industrial exige uso industrial o un edificio industrial o depósito, con un puntaje de al menos 2 y no menor que el comercial. Si no hay señal, el tramo queda residencial.

Sobre las 1257 calles hay 859 residenciales (68 %) y 398 comerciales (32 %). Industrial es 0 %.

## 2.5. Columnas generadas

Días sin recolección, quejas vecinales y la prioridad no están en los datos del GCBA. Las genera un script con el generador de numpy.

Otras dos columnas no entran en la fórmula de la prioridad: si el tramo está cerca de una escuela o un hospital, y si hay feria o evento. Las agrega `notebooks/tramos.ipynb`, con un sorteo de Bernoulli cuya probabilidad depende de la zona y no es la misma para las dos columnas. En el CSV, cerca del 8 % de los tramos marca cada una. El modelo las recibe igual.

### 2.5.1. Días y quejas

Los días salen de un Poisson. La media depende de la zona y sube con la cantidad de contenedores: 3,2 en residencial, 2,0 en comercial y 4,5 en industrial, más 0,15 por cada contenedor. El resultado se recorta entre 0 y 10.

Las quejas también salen de un Poisson. La tasa crece con los contenedores, con los días sin recolección y en zona comercial:

$$\begin{aligned}\lambda = \exp(&-0{,}4 + 0{,}35 \cdot \log(1 + \text{contenedores}) \\ &+0{,}12 \cdot \text{días} + b_{\text{zona}})\end{aligned}$$

El bonus de zona es 0 en residencial, 0,55 en comercial y 0,15 en industrial.

Sin contenedor no hay una recolección que medir en ese tramo, así que los días y las quejas quedan en 0. Con al menos un contenedor, los dos conteos quedan en 1 como mínimo.

### 2.5.2. La prioridad

La etiqueta es un corte por cantidad de filas. Primero se arma un puntaje:

$$\begin{aligned}\text{puntaje} = {} & 0{,}55 \cdot \text{días} + 0{,}70 \cdot \log(1 + \text{contenedores}) \\ & + 0{,}45 \cdot \log(1 + \text{quejas}) + b_{\text{zona}} \\ & + 0{,}18 \cdot \text{días} \cdot \log(1 + \text{contenedores}) + \text{ruido}\end{aligned}$$

El bonus de zona de este puntaje es 0 en residencial, 0,40 en comercial y 0,20 en industrial. El ruido es normal, de desvío 0,40. Los tramos se ordenan de mayor a menor puntaje. El 20 % de arriba queda en alta, el 35 % siguiente en media y el resto en baja.

En una calle sin contenedores los días y las quejas son 0, así que el puntaje es el bonus de zona más el ruido. Dos calles con los mismos datos pueden caer una en baja y otra en media. Esa superposición aparece después en los errores del modelo.

### 2.5.3. Cómo se generaron

Días sin recolección, quejas y prioridad los escribe `backend/dataset/synthesize.py`. La semilla está en `backend/config.py` y vale 42, así que volver a correr el script deja las mismas columnas. `backend/dataset/build_dataset.py` llama a esa función al armar el CSV. Calles y contenedores siguen saliendo de OpenStreetMap y del CSV del GCBA.

Sin contenedor no hay una recolección que medir en esa calle, y la función deja días y quejas en 0; con al menos un contenedor, los dos conteos quedan en 1 como mínimo. `backend/dataset/resynthesize.py` vuelve a correr esa misma función sobre el CSV ya armado, sin tocar las columnas de escuela u hospital ni de feria, y recalcula la prioridad.

# Capítulo 3

# El modelo

El clasificador predice la prioridad de cada tramo. Se comparan regresión logística, árbol de decisión, k vecinos más cercanos y SVM. La comparación está en `notebooks/modelos.ipynb`. El modelo elegido se mide una sola vez, en filas que no entraron en la elección.

## 3.1. Columnas

Las features de entrada son seis columnas: días sin recolección, contenedores, quejas, tipo de zona, cerca de escuela u hospital, y feria o evento. La etiqueta es `prioridad_recoleccion`; no es una feature.

Las cuatro primeras entran porque con ellas se arma la etiqueta. Las otras dos son contexto del tramo; la fórmula de la prioridad no las usa.

El identificador, el nombre de la calle, la distancia y el tipo de vía quedan afuera: no describen la urgencia de recolectar.

## 3.2. Comparación

El 20 % de las filas queda aparte para el test y no entra en la elección. El corte es estratificado, así alta, media y baja mantienen su proporción. Son 1005 filas para entrenar y elegir, y 252 para el test.

Ese 80 % se parte en cinco. Cada candidato se entrena cinco veces y cada vez valida con una parte distinta. El número para comparar es el promedio del F1 macro. El desvío muestra cuánto se mueve ese promedio entre las cinco partes.

El F1 macro pesa las tres clases por igual. Baja es el 45 % de las filas, y el accuracy puede quedar alto acertando casi siempre esa clase. Una baseline que predice siempre la clase más frecuente marca el piso de esa comparación.

| | modelo | parametros | f1_macro | desvio |
|---|---|---|---|---|
| 0 | Logística | max_iter=5000 | 0.779 | 0.009 |
| 1 | SVM | {'C': 1, 'kernel': 'linear'} | 0.776 | 0.011 |
| 2 | Árbol | {'max_depth': 5, 'min_samples_leaf': 5} | 0.760 | 0.009 |
| 3 | kNN | {'n_neighbors': 13} | 0.743 | 0.032 |
| 4 | Baseline (siempre la más frecuente) | — | 0.207 | 0.001 |

Figura 3.1: F1 macro promedio en validación cruzada por modelo. La baseline marca el piso.

La regresión logística usa el ajuste por defecto, con un máximo de 5000 iteraciones. El árbol varía la profundidad y el mínimo de hojas, y no se escala. kNN varía la cantidad de

vecinos, solo los impares. SVM varía el margen y el tipo de frontera, recta o curva. Logística, kNN y SVM escalan las columnas numéricas, para que la de números más grandes no pese solo por la escala. El tipo de zona pasa a columnas de ceros y unos. Escalar y codificar se ajustan solo con las filas de entrenamiento de cada ronda.

## 3.3. Resultado

Con el F1 macro de esa validación, la regresión logística queda primera. El SVM lineal queda muy cerca, dentro del desvío. El SVM de frontera curva, el árbol y kNN quedan por debajo. La baseline queda lejos de los cuatro.

El modelo que carga el servidor es la logística, con el ajuste por defecto y el tope de 5000 iteraciones.

En el test llega a 0,754 de accuracy y 0,755 de F1 macro. Esas dos salen de la matriz de abajo. El soporte es 50 tramos altos, 88 medios y 114 bajos: 252 filas.

$$\text{precisión} = \frac{\text{VP}}{\text{VP} + \text{FP}} \qquad \text{recall} = \frac{\text{VP}}{\text{VP} + \text{FN}}$$

$$F_1 = \frac{2 \cdot \text{precisión} \cdot \text{recall}}{\text{precisión} + \text{recall}}$$

$$\text{accuracy} = \frac{45 + 44 + 101}{252} = 0{,}754 \qquad F_{1,\text{macro}} = \frac{0{,}874 + 0{,}587 + 0{,}805}{3} = 0{,}755$$

El F1 macro es el promedio de los tres F1. El accuracy cuenta aciertos sobre 252 y premia acertar la clase grande: baja concentra 114 de 252. La baseline que siempre predice baja alcanza accuracy 0,452 y F1 macro 0,208, porque el F1 de alta y el de media son 0. Por eso el modelo se elige con F1 macro: las tres clases pesan igual.

Alta: VP 45, FP 8, FN 5. Precisión $45/53 = 0{,}849$. Recall $45/50 = 0{,}900$. F1 0,874.

Media: VP 44. Los FP son 5 más 13, en total 18. Los FN son 8 más 36, en total 44. Precisión $44/62 = 0{,}710$. Recall $44/88 = 0{,}500$. F1 0,587.

Baja: VP 101, FP 36, FN 13. Precisión $101/137 = 0{,}737$. Recall $101/114 = 0{,}886$. F1 0,805.

La baseline del test es un `DummyClassifier` con estrategia `most_frequent`: siempre predice baja. En la validación cruzada ese piso de F1 macro queda en 0,207.

Por clase:

| | precisión | recall | F1 |
|---|---|---|---|
| alta | 0,849 | 0,900 | 0,874 |
| media | 0,710 | 0,500 | 0,587 |
| baja | 0,737 | 0,886 | 0,805 |

El test tiene 50 tramos altos, 88 medios y 114 bajos. Alta se predice bien. En media, la mitad de los tramos del test no se reconoce, y muchos de esos quedan marcados como baja.

| | predicha alta | predicha media | predicha baja |
|---|---|---|---|
| real alta | 45 | 5 | 0 |
| real media | 8 | 44 | 36 |
| real baja | 0 | 13 | 101 |

Cuadro 3.1: Matriz de confusión del test con regresión logística.

La matriz cuenta tramos del test. La fila es la clase real y la columna es la clase predicha. La diagonal son aciertos.

El modelo se confunde entre media y baja. De los 88 tramos medios del test, 36 quedan marcados como baja, y 13 tramos bajos quedan marcados como media. Alta y baja no se cruzan: ningún tramo alto se predice bajo, ni al revés. Esos 36 hacen que la precisión de baja (0,737) quede debajo de su recall (0,886): el modelo marca como baja varias calles que eran medias.

Creemos que el motivo está en las calles sin contenedor. De los 1257 tramos, 848 no tienen ninguno y, por la regla del generador, tienen días 0 y quejas 0. El puntaje de esas filas es el bonus de zona más el ruido, y el corte por ranking las reparte entre baja y media. Las mismas entradas quedan con dos etiquetas distintas, y el modelo no tiene otra señal para separarlas. Donde hay contenedor, días y quejas, el puntaje distingue mejor y alta se predice con holgura.

La API usa ese ajuste. El circuito se arma con las calles que el modelo marca como alta. Confundir media con baja no cambia ese conjunto ni el largo de las calles, así que no mueve el recorrido. Sí lo mueve un tramo que pasa de alta a media, o al revés: en el test son 5 altos marcados como media y 8 medios marcados como alta. Si se edita una calle y la predicción cruza ese corte, el recorrido se calcula de nuevo.

# Capítulo 4

# El circuito

La salida del modelo es una prioridad por tramo. El camión tiene que pasar por todas las calles de prioridad alta. Las de prioridad media y baja entran solo si quedan en el camino que une esas calles.

## 4.1. Calles a cubrir

El subgrafo de las calles altas queda en varias componentes, y en algunas esquinas llega una cantidad impar de esas calles. Un circuito euleriano pide un grafo conexo y grados pares en todas las esquinas. Hierholzer recorre cada arista una vez y vuelve al inicio cuando esas dos condiciones se cumplen. Si no se cumplen, se agregan copias de calles hasta dejar el multigrafo en esas condiciones.

Esas copias son el empalme: el tramo que el camión recorre para pasar de una calle alta a otra. El costo del circuito suma los metros de las calles altas y los metros de los empalmes.

## 4.2. El camino de unión

Para pasar de una componente a otra se busca el camino más corto, en metros, sobre el grafo completo. Lo calcula Dijkstra y el peso de cada calle es su largo. Si ese camino pasa por una calle media o por una baja, el camión la recorre igual: la prioridad no cambia el peso.

## 4.3. Grados impares

Con esquinas de grado impar en el subgrafo de las altas no hay un circuito que pase por cada calle alta una sola vez. La cantidad de esas esquinas es par. Se calculan las distancias mínimas entre todas ellas con Dijkstra, sobre el grafo completo, y se emparejan de a dos de modo que la suma de esas distancias sea la menor posible. Cada pareja aporta su camino mínimo, y ese camino se copia una vez. Las dos esquinas impares pasan a tener grado par.

El emparejamiento lo resuelve rustworkx con el algoritmo de Edmonds. La librería busca el peso máximo, así que cada distancia entra negada. Los metros pasan a milímetros porque el peso tiene que ser entero.

La primera versión usaba `networkx.min_weight_matching`. Es el mismo algoritmo de Edmonds, escrito en Python. Con 318 esquinas impares el emparejamiento arma un grafo completo de 50403 aristas, y el costo crece con el cubo de la cantidad de esquinas. En esa corrida el emparejamiento tardó 16,9 segundos, de los 17 que tardaba el circuito entero. rustworkx corre el mismo procedimiento en Rust. El emparejamiento pasó a 0,16 segundos y

el circuito completo a 0,7. El largo de los caminos elegidos no cambió: 10780 metros en las dos versiones.

## 4.4. Componentes que siguen separadas

El emparejamiento une componentes cuando el camino entre dos esquinas impares cruza de una a otra. Las que siguen separadas, y también las que ya tenían todos los grados pares, se unen con el camino mínimo entre ellas.

Ese empalme se agrega dos veces. Una sola copia dejaría las dos puntas con grado impar. Ida y vuelta mantiene los grados y conecta las componentes. Los empalmes siguen un árbol: se unen las componentes más cercanas y no se cierra un ciclo entre ellas. El multigrafo queda conexo. Si las calles altas ya formaban una sola componente, este paso no agrega calles.

Cuando el subgrafo de las altas es conexo, este armado es el cartero chino. Cuando no lo es, primero se emparejan los grados impares y después se unen las componentes que siguen sueltas con un árbol de caminos mínimos. Ese segundo paso conecta lo que el emparejamiento dejó separado; no busca, entre todos los circuitos que cubren las altas, el de menor largo.

## 4.5. El recorrido

Con el multigrafo conexo y con todos los grados pares, Hierholzer arma el circuito. La secuencia de esquinas empieza y termina en el mismo nodo. Cada copia de calle se usa una vez. Si una calle aparece dos veces en la lista, había dos copias y el camión la recorre en dos momentos del viaje.

La aplicación muestra esa lista en orden, con el largo de cada paso y el largo total. Las calles que se recorren más de una vez quedan marcadas. El circuito vuelve al punto de partida y pasa al menos una vez por cada calle de prioridad alta.

# Capítulo 5

# Capturas del frontend

La aplicación está publicada en https://paz-reyes.moloeznik.me/. El mapa carga el microcentro. El color sigue la prioridad que predice el modelo: negra alta, naranja media, gris baja.

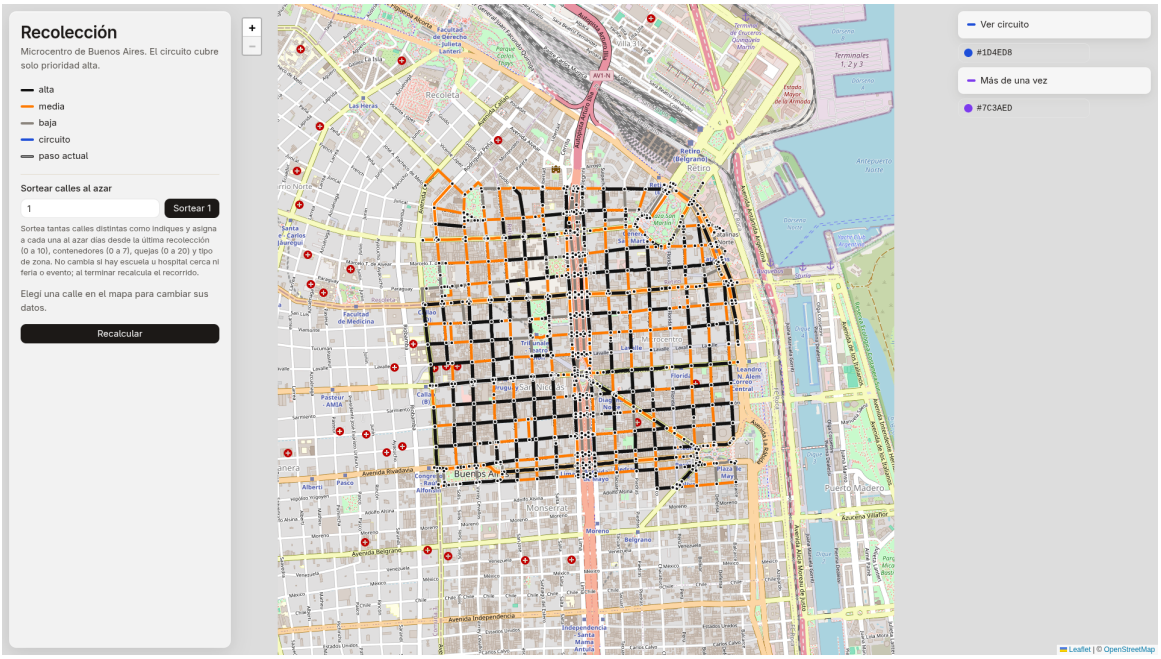



Figura 5.1: Página principal: el grafo de calles, sin el circuito dibujado.

Al pulsar *Ver circuito* se calcula el recorrido y se dibuja en azul. Con *Mostrar solo el circuito* el panel lista el costo y los pasos. En esa vista el recorrido mide 40,8 km y cubre 254 calles altas. También se puede elegir una calle y cambiar sus datos de a una.

El sorteo elige tantas calles distintas como se indique, les asigna días, contenedores, quejas y zona al azar, y recalcula. Cerca de escuela u hospital, y feria o evento, se copian. Con 80 calles el recorrido pasa a 46,0 km y 312 calles altas. El trazo deja de ser el de antes.

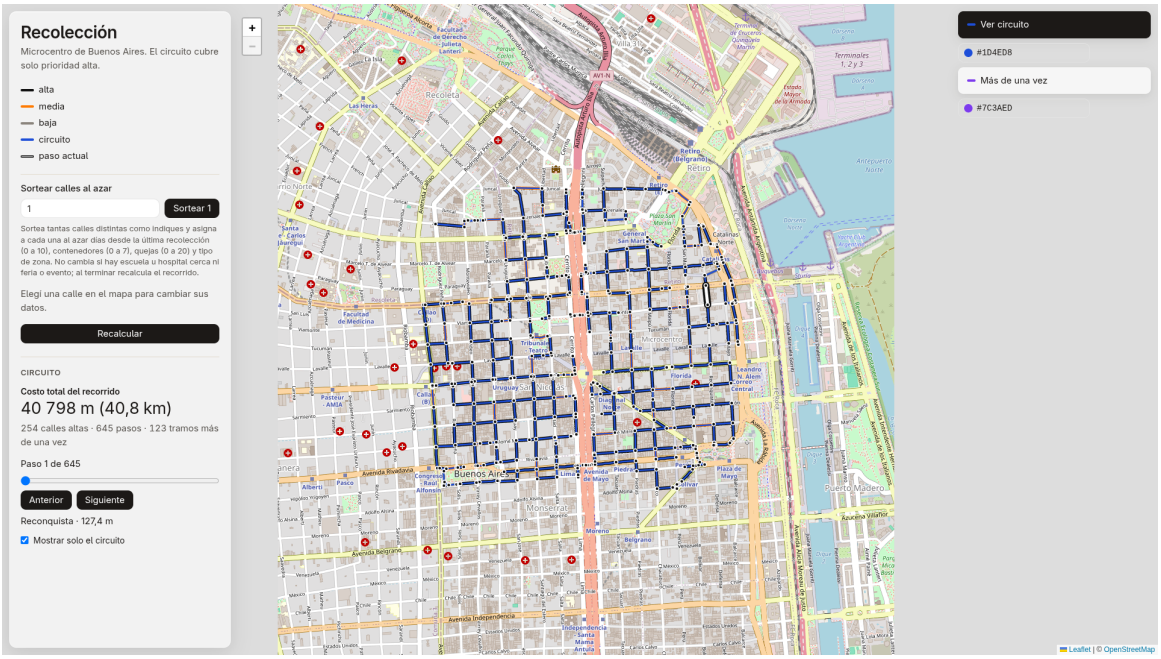



Figura 5.2: Circuito del camión, con el costo y el paso actual.

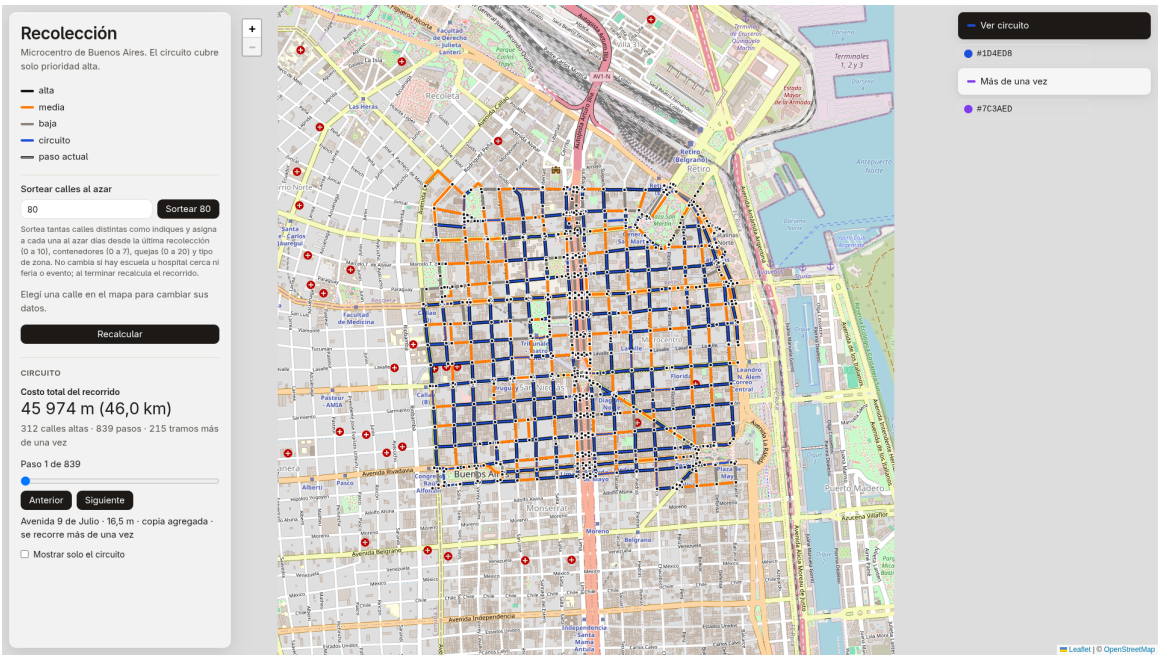



Figura 5.3: Después de sortear 80 calles y recalcular el circuito.

# Capítulo 6

# Uso de inteligencia artificial

## 6.1. Qué herramientas se usaron

Pendiente.

## 6.2. Para qué partes se usaron, y para cuáles no

Pendiente.

## 6.3. Ejemplos de prompts

Pendiente.

## 6.4. Casos en los que la IA se equivocó

Pendiente.

## 6.5. Reflexión

Pendiente.

# Capítulo 7

# Limitaciones

El mapa es el microcentro. De los 2292 contenedores de la zona 1, 1405 quedan a más de 50 metros de las calles de ese recorte y no entran. Toda la zona 1 se iba del tope de unas 1500 aristas con el que se armó el grafo.

El modelo confunde medias con bajas, y ese intercambio no cambia el recorrido: el circuito solo obliga a pasar por las altas y el camino de unión se mide en metros. Lo que mueve el circuito es el cruce entre alta y media, que en el test son 5 altos marcados como media y 8 medios marcados como alta.